Problem solving by searching

AIMA Chapter 3

What are Search Problems?

- **Problem solving by searching** is the process of finding a sequence of actions that transforms an initial state into a desired goal state by systematically exploring the possible states of the problem (state space).
- **Why is Searching Needed?**
- Many real-world problems do not have a direct formula or algorithm.
- Examples:
  - Finding the shortest route on Google Maps.
  - Solving a Rubik's Cube.
  - Playing Chess.
  - Robot navigation.
  - Puzzle solving (8-puzzle, 15-puzzle).

- The agent must **search** among many possibilities before deciding what to do.
- We will consider the problem context of designing **goal-based agents** in **known**, **fully observable, static and and deterministic** environments.

The 5 Components of a Search Problem

To solve a problem via search, the system must first translate the real-world scenario into a mathematically defined "state space" using five parameters:

- **Initial State**: The starting point or current condition of the environment (e.g., your current location on a map).
- **Actions**: The complete set of legal moves available to the agent at any given point (e.g., turning left, turning right, or continuing straight).
- **Transition Model**: A formal description of what each action does (e.g., if you are at Intersection A and take the action "go east," you arrive at Intersection B).
- **Goal Test**: A condition that determines whether a specific state is the final objective (e.g., checking if the current coordinates match the destination).
- **Path Cost**: A function that assigns a numeric cost to a sequence of actions. It helps the agent find the *optimal* solution rather than just *any* solution (e.g., minimizing travel time, fuel consumed, or distance).

For the classic Water Jug Problem with a 4-liter jug and a 3-liter jug, the problem formulation is:

Component	Formulation
State	((x,y)), where (x) is water in the 4-L jug and (y) is water in the 3-L jug
Initial State	((0,0)) — both jugs are empty
Actions	Fill either jug, empty either jug, or pour water from one jug to the other
Transition Model	Describes the new state resulting from each action
Goal Test	Obtain exactly **2 liters** of water, e.g. (x=2 or y=2)
Path Cost	Usually **1 per action**; minimize the number of actions

Example: Romania Vacation

- On vacation in Romania; currently in Arad
- Flight leaves tomorrow from Bucharest

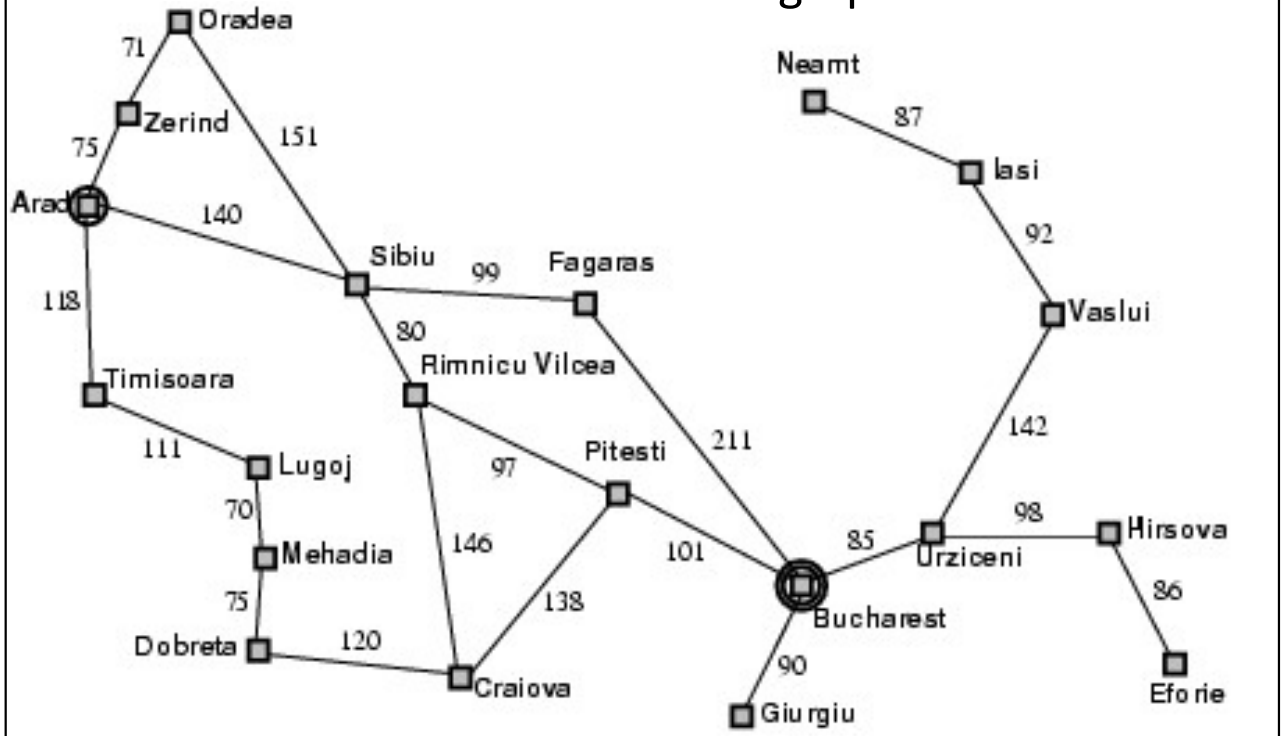
Original Description



- **Initial state:** Arad
- **Actions:** Drive from one city to another.
- **Transition model and states:** If you go from city A to city B, you end up in city B.
- **Goal state:** Bucharest
- **Path cost:** Sum of edge costs.

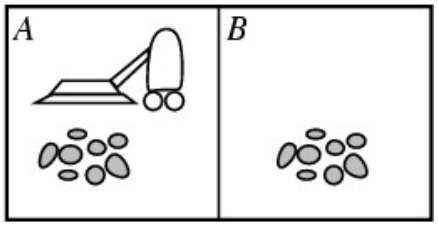
State Space/Transition model

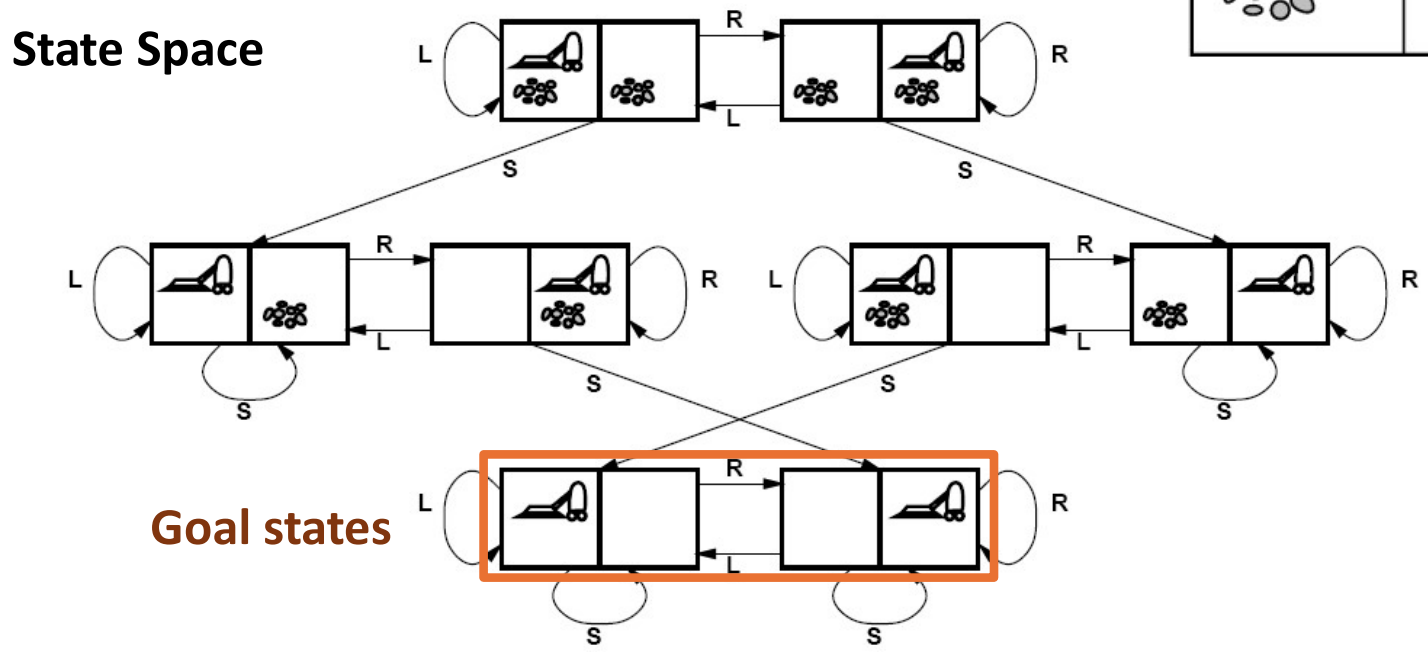
Defined as a graph



Distance in miles

Example: Vacuum world

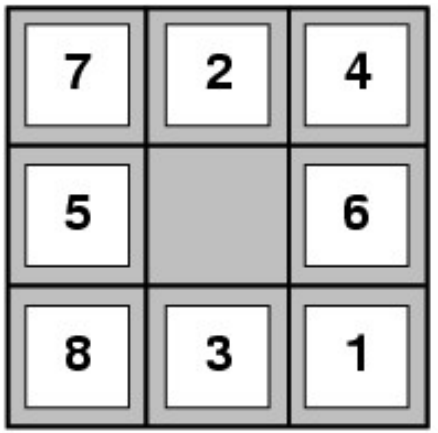




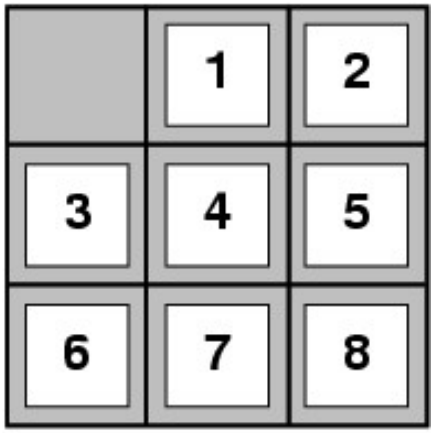
- **Initial State:** Defined by agent location and dirt location.
- **Actions:** Left, right, suck
- **Transition model:** Clean a location or move.
- **Goal state:** All locations are clean.
- **Path cost:** E.g., number of actions

Example: Sliding-tile Puzzle

- **Initial State:** A given configuration.
- **Actions:** Move blank left, right, up, down
- **Transition model:** Move a tile
- **Goal state:** Tiles are arranged empty and 1-8 in order
- **Path cost:** 1 per tile move.



Start State



Goal State

Searching for Solutions

- Once a problem is formulated, the agent uses a **search graph** to find a solution. The root of the tree represents the initial state, and the branches represent actions.
- The process distinguishes between two main types of search algorithms based on how much information they possess:
  - Uninformed Search Strategies (Blind Search)
  - Informed Search Strategies (Heuristic Search)

A. Uninformed Search Strategies (Blind Search)

- These algorithms have no information about the cost or distance from the current state to the goal. They can only generate successors and distinguish a goal state from a non-goal state.
- **Breadth-First Search (BFS):** Expands the shallowest unexpanded node first.
- **Depth-First Search (DFS):** Expands the deepest unexpanded node first.
- **Depth-Limited & Iterative Deepening Search (IDS):** IDS combines the benefits of DFS's space efficiency with BFS's completeness and optimality by gradually increasing the depth limit.
- **Uniform-Cost Search (UCS):** Expands the node with the lowest path cost , g(n). It is optimal for any step-cost distribution.
- **Bidirectional search:** Bidirectional search works by simultaneously running two searches—one forward from the start state and one backward from the goal state—until their paths intersect in the middle.

B. Informed Search Strategies (Heuristic Search)

- These algorithms use domain-specific knowledge via a **heuristic function**, denoted as h(n), which estimates the cost of the cheapest path from node n to a goal state.
- **Greedy Best-First Search:** Expands the node that appears closest to the goal based purely on the heuristic (f(n) = h(n)).
- **A* Search:** The most widely known form of best-first search. It evaluates nodes by combining the cost to reach the node (g(n)) and the cost to get from the node to the goal (h(n)):

$$f(n) = g(n) + h(n)$$

Tree search vs Graph search

- The core difference between **Tree Search** and **Graph Search** is how they handle **redundant paths** and **repeated states**.
- While Tree Search can get trapped exploring the same states over and over again, Graph Search remembers where it has already been to avoid wasting time.
- In Graph Search, the algorithm relies on the interaction between two distinct data structures: the **Frontier** (or Open List) and the **Explored Set** (or Closed List).
- Graph Search tracks states using the **Frontier** (Open List) and the **Explored Set** (Closed List) to eliminate redundant paths and infinite loops.

The General Search Algorithm (Graph Search)

The general search algorithm maintains two sets of nodes:

- **The Frontier (Open List):** The set of all leaf nodes available for expansion.
- **The Explored Set (Closed List):** The set of all states that have already been expanded, preventing the algorithm from getting stuck in infinite loops.

Pseudo-code for Late Goal Testing

```
function GENERAL-GRAPH-SEARCH(problem) returns a solution, or failure
    initialize the frontier using the initial state of problem
    initialize the explored set to be empty
    loop do
        if the frontier is empty then return failure
        choose a node and remove it from the frontier according to the search strategy
        if the node contains a goal state then return the corresponding solution
        add the node's state to the explored set
        for each action in problem.ACTIONS(node.STATE) do
            child <- CHILD-NODE(problem, node, action)
            if child.STATE is not in explored or frontier then
                insert child into the frontier
            else if this is Uniform-Cost Search and child.STATE is in frontier with higher cost
                    then replace that frontier node with child
```

Evaluating Search Performance

Metric	Description
Completeness	Is the algorithm guaranteed to find a solution if one exists?
Optimality	Does the strategy find the highest-quality solution (lowest path cost)?
Time Complexity	How long (or how many nodes generated) does it take to find a solution?
Space Complexity	How much memory does the algorithm need to perform the search?

Note: Complexity is typically expressed in terms of b, the branching factor; d, the depth of the shallowest goal; and m, the maximum depth of the state space.

Breadth-First Search (BFS)

Expansion rule: Expand shallowest unexpanded node in the frontier (=**FIFO**).

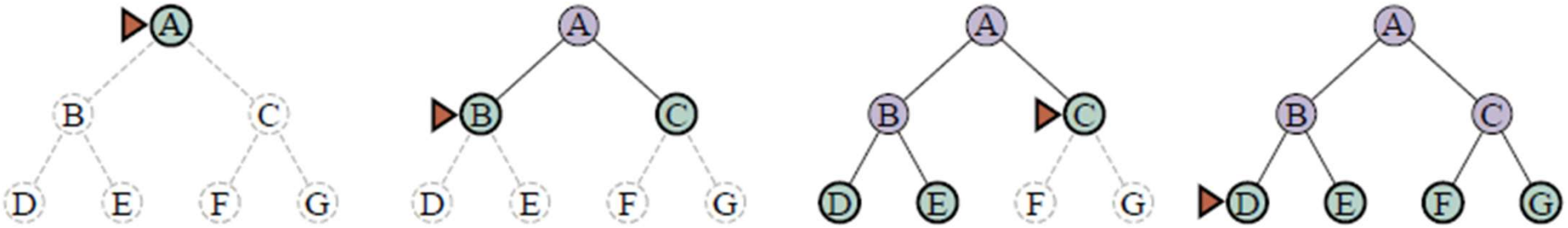


Figure 3.8 Breadth-first search on a simple binary tree. At each stage, the node to be expanded next is indicated by the triangular marker.

Data Structures

- **Frontier** data structure: holds references to the green nodes (green) and is implemented as a FIFO **queue**.
- **Reached** data structure: holds references to all visited nodes (gray and green) and is used to prevent visiting nodes more than once (cycle and redundant path checking).
- Builds a **complete tree** with links between parent and child.

Breadth-First Search (BFS)

BFS expands the shallowest nodes first, exploring the search tree layer by layer.

- **Completeness: Yes.** If the shallowest goal node is at a finite depth d, BFS will eventually find it, provided the branching factor b is finite.
- **Optimality: Yes**, but *only* if all step costs are identical/equal (e.g., every action costs 1). If step costs vary, the shallowest goal might not be the cheapest one.
- **Time Complexity: O(b^d)**. In the worst case, the algorithm processes every node up to the goal depth.
- **Space Complexity: O(b^d)**. This is BFS's biggest flaw. It must keep every generated node in memory (the FIFO queue), which means memory boundaries are usually hit long before time limits become an issue.

Exponential Space (and time) Not Good…

- Exponential complexity uninformed search problems *cannot* be solved for any but the smallest instances.
- *(Memory* requirements are a bigger problem than *execution* time.)

DEPTH	NODES	TIME	MEMORY
2	**110**	**0.11 milliseconds**	**107 kilobytes**
4	**11110**	**11 milliseconds**	**10.6 megabytes**
6	**10^6**	**1.1 seconds**	**1 gigabytes**
8	**10^8**	**2 minutes**	**103 gigabytes**
10	**10^{10}**	**3 hours**	**10 terabytes**
12	**10^{12}**	**13 days**	**1 petabytes**
14	**10^{14}**	**3.5 years**	**99 petabytles**

Assumes b=10, 1M nodes/sec, 1000 bytes/node

Depth-First Search (DFS)

- **Expansion rule**: Expand deepest unexpanded node in the frontier (last added).
- **Frontier**: **stack** (LIFO)
- **No reached data structure:** forgets completely explored subtrees.
- **Needs Cycle checking:** don't expand nodes that are already in the current path to the root node.
- **Cannot avoid redundant paths:** Leads to multiple nodes representing the same state and replicated work.

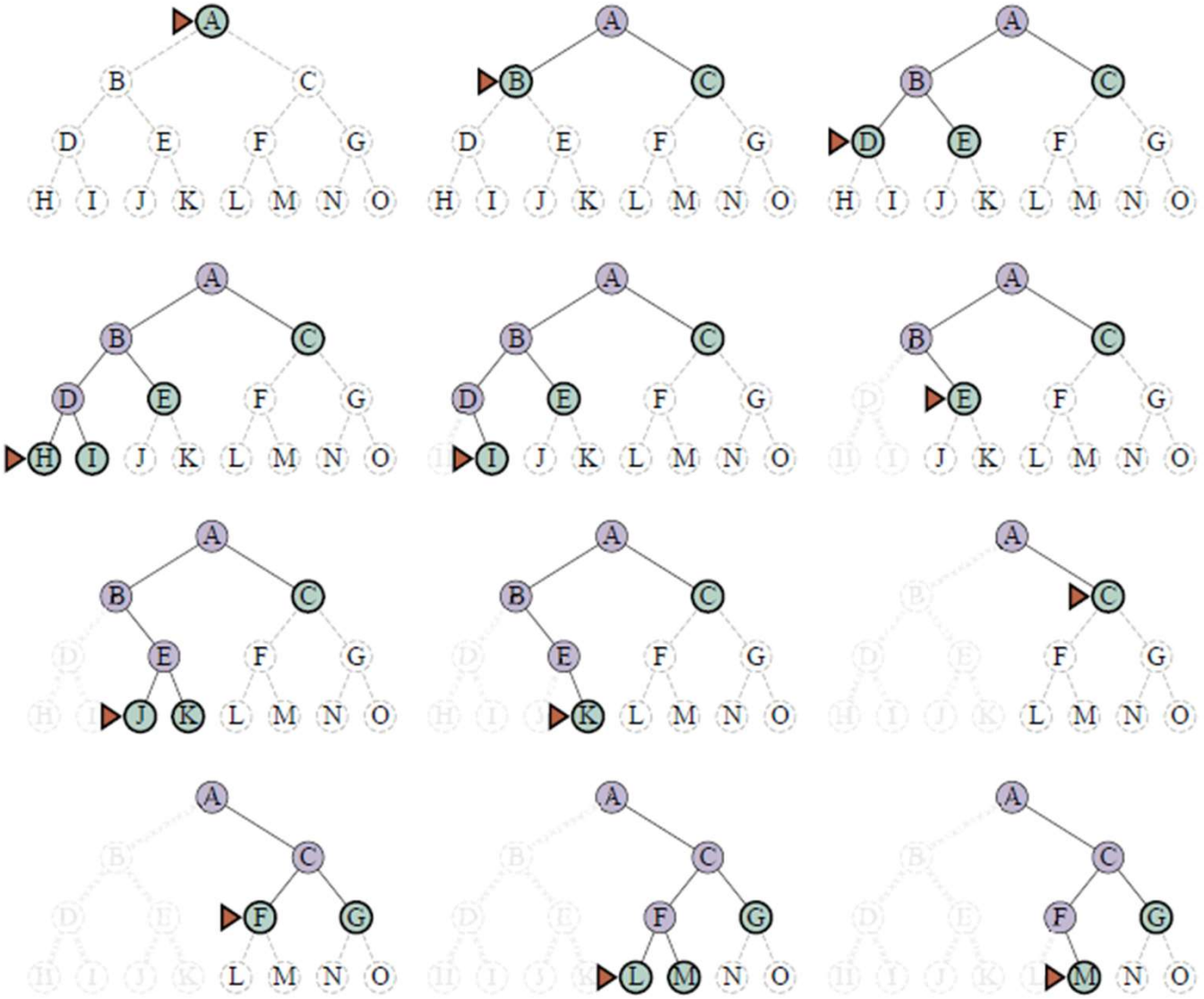


Figure 3.11 A dozen steps (left to right, top to bottom) in the progress of a depth-first search on a binary tree from start state A to goal M. The frontier is in green, with a triangle marking the node to be expanded next. Previously expanded nodes are lavender, and potential future nodes have faint dashed lines. Expanded nodes with no descendants in the frontier (very faint lines) can be discarded.

Depth-First Search (DFS)

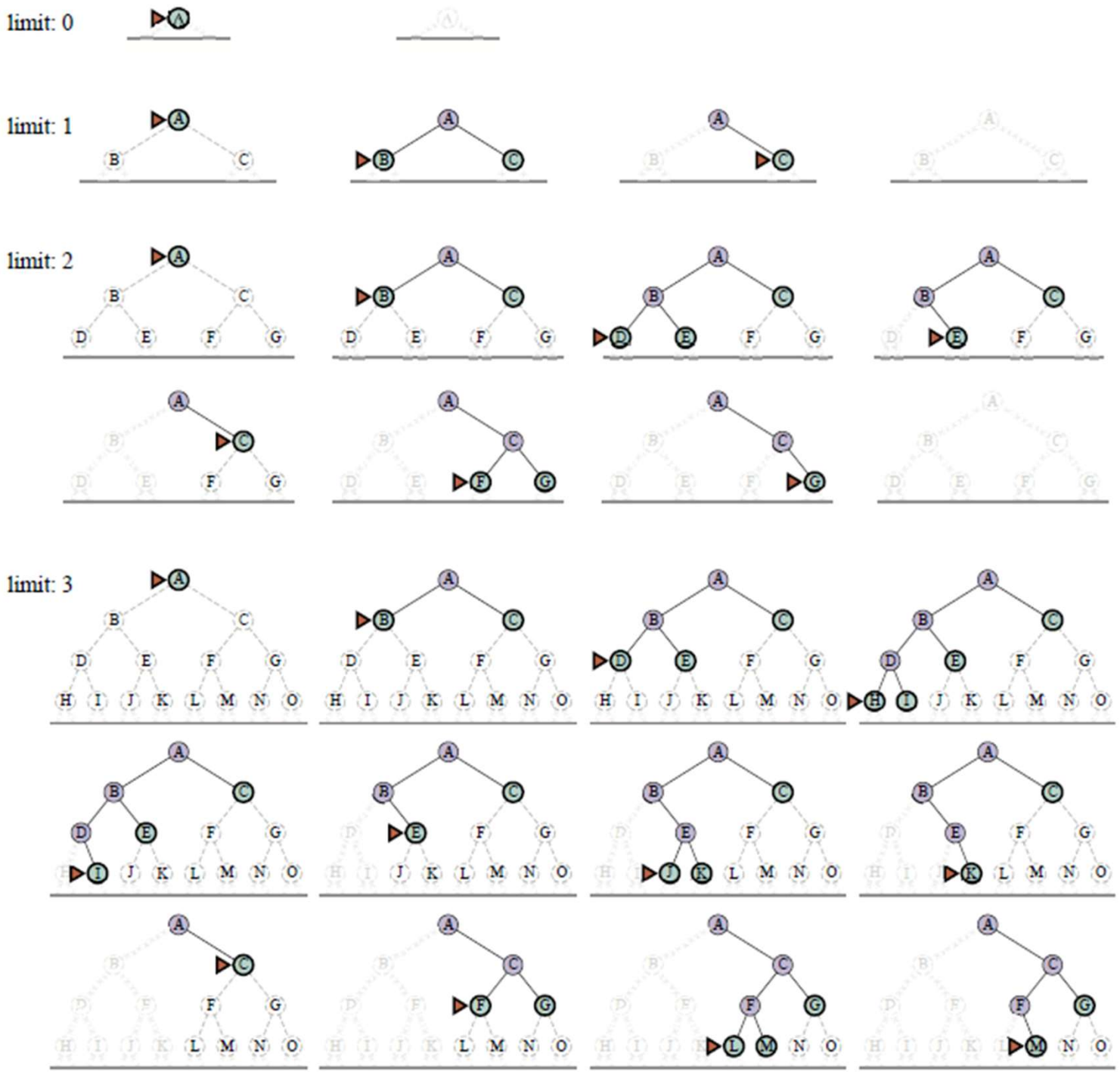
DFS expands the deepest unexpanded node first, moving down a single path until it hits a dead end, using a LIFO queue (stack).

- **Completeness:**
  - **No** in infinite-depth spaces (if a branch goes down forever without a goal, DFS will get stuck).
  - **Yes** in finite state spaces *if* graph search is used (to avoid loops).
- **Optimality: No.** DFS returns the *first* solution it encounters, which might be a deep, high-cost path even if a shallow, cheap solution exists on a different branch.
- **Time Complexity: O(b^m)**. In the worst-case scenario, DFS might have to visit every single node in the entire state space down to the maximum depth m. If m is much larger than d, this is highly inefficient.
- **Space Complexity: O(bm)**. This is DFS's primary advantage. It only needs to store the single path from the root to the current node, along with the remaining unexpanded sibling nodes for each node on the path.

Depth-Limited Search (DLS)

- DLS is DFS with a pre-determined depth limit l. Nodes at depth l are treated as if they have no successors.
- **Completeness: No** if the choose limit l < d (the goal is deeper than the cutoff). It is only complete if l >= d.
- **Optimality: No.** Like DFS, it does not guarantee finding the shallowest or cheapest goal first.
- **Time Complexity: O(b^l)**. It explores up to the maximum forced depth limit l.
- **Space Complexity: O(bl)**. It retains the space efficiency of standard DFS, bounded by the limit l.

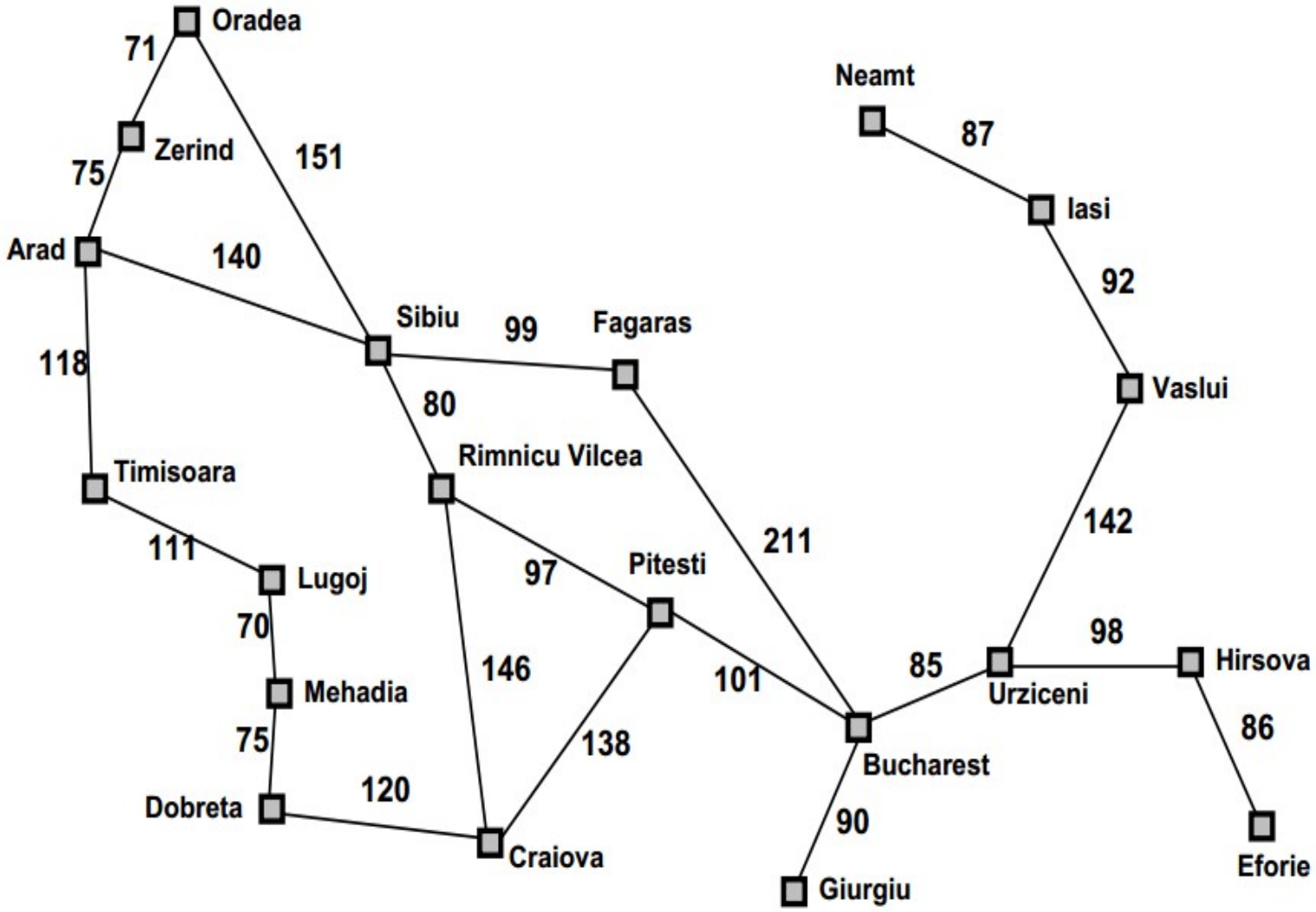
Iterative Deepening Search (IDS)



Iterative Deepening Deep-First Search (IDS)

- IDS repeatedly applies Depth-Limited Search, gradually increasing the depth limit l (l = 0, 1, 2, …) until a goal is found.
- **Completeness: Yes**, as long as the branching factor b is finite. It behaves like BFS by systematically checking every level.
- **Optimality: Yes**, exactly like BFS, it is optimal if all step costs are equal.
- **Time Complexity: O(b^d)**. While it seems wasteful to re-generate upper-level nodes repeatedly, the overhead is surprisingly small. In an exponential tree, the vast majority of nodes are at the bottom layer, so the math works out to roughly the same order of magnitude as BFS.
- **Space Complexity: O(bd)**. This is why IDS is often the preferred uninformed search strategy for large state spaces. It combines the **optimality and completeness of BFS** with the **minimal memory footprint of DFS**.

Romania with step costs in km



Straight–line distance to Bucharest

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Uniform-Cost Search (UCS)

- UCS expands the node n with the lowest path cost g(n) instead of the shallowest node. It implements a priority queue sorted by g(n).
- **Completeness: Yes**, provided that the cost of every step is greater than or equal to a small positive constant. This prevents the algorithm from getting stuck in infinite loops of zero-cost actions.
- **Optimality: Yes.** Nodes are expanded in strictly increasing order of their optimal path cost. The first goal node chosen for expansion is guaranteed to be the optimal one.
- **Time & Space Complexity:**

$$O\left(b^{1+\lfloor C^*/\epsilon \rfloor}\right)$$

- where C* is the cost of the optimal solution. Instead of being bounded by depth d, UCS is guided by the actual cost and let the minimum cost of any action be: $\epsilon > 0$.
- So C^*/ϵ gives an **upper bound on how deep a node with cost no more than C^* can be**.
- When all step costs are equal, this simplifies right back to O(b^d).

Let the **minimum cost of any action** be:

$$\epsilon > 0$$

If a path contains d steps, and every step costs at least ϵ, then:

$$g(n) \geq d\epsilon$$

For a node to have cost at most C^*:

$$d\epsilon \leq C^*$$

Therefore:

$$d \leq \frac{C^*}{\epsilon}$$

Since depth must be an integer:

$$\boxed{d_{\max} = \left\lfloor \frac{C^*}{\epsilon} \right\rfloor}$$

So C^*/ϵ gives an **upper bound on how deep a node with cost no more than C^* can be.**

Informed Search

AI search problems typically have a very large search space. We would like to improve efficiency by **expanding as few nodes as possible.**

Idea: The agent can use **additional information** in the form of "hints" about what promising states are to explore first. These hints are derived from

- **information** the agent has (e.g., a map with the goal location marked) or
- **percepts** coming from a sensor (e.g., a GPS sensor and coordinates of the goal).

Method: The agent uses a **heuristic function $h(n)$**

- to rank nodes in the frontier based on the additional information, and
- to select the most promising node in the frontier for expansion using the **best-first search** strategy.
- **Heuristic function** $h(n)$ estimates the cost of reaching a node representing the goal state from the currently considered node n

Discussed algorithms:

- Greedy best-first search
- A* search

Heuristic for the Romania Problem

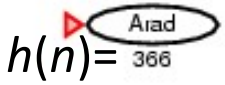
Use the map for hints: Estimate the driving distance from Arad to Bucharest using a straight-line distance on the map.

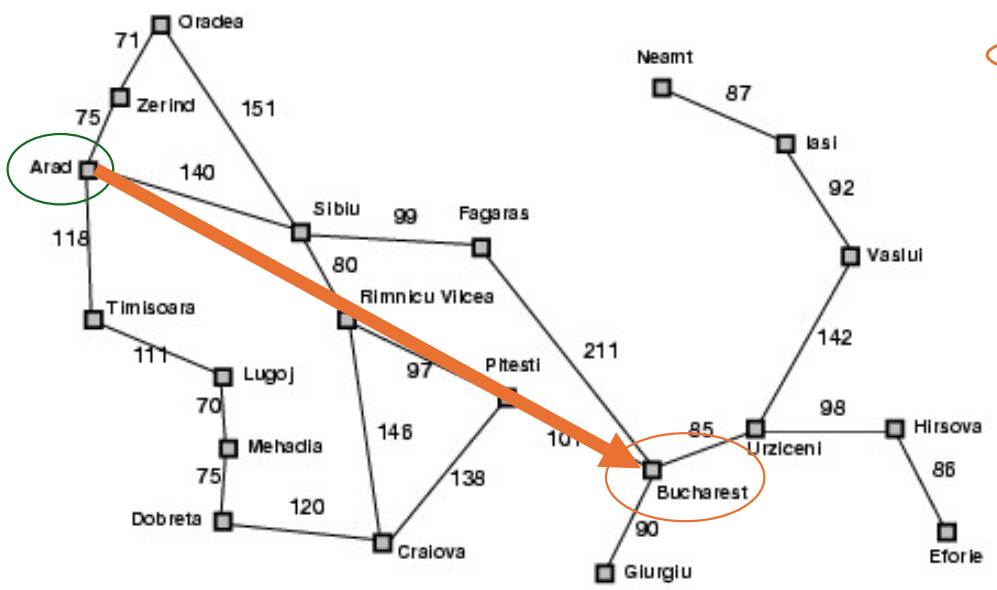


Straight-line distance to Bucharest	$h(n)$
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Greedy Best-First Search Example

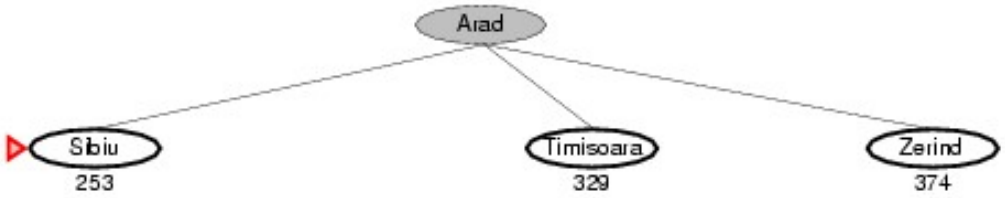
Expansion rule: Expand the node that has the lowest value of the heuristic function *h*(*n*)

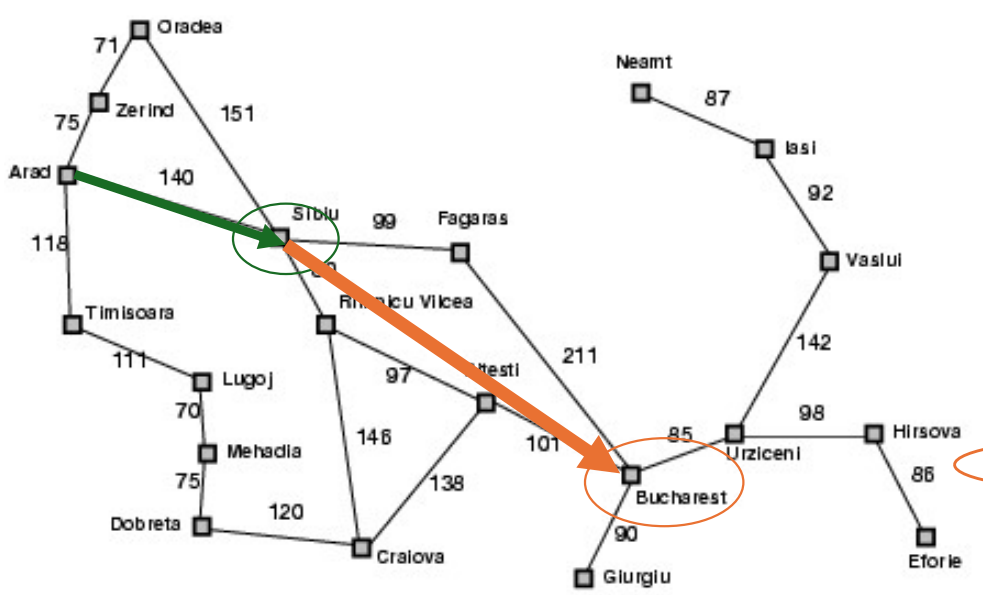




Straight–line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

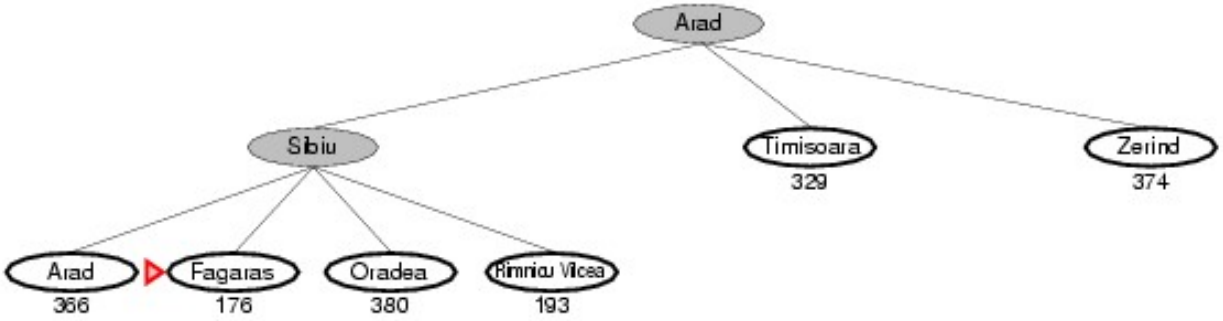
Greedy Best-First Search Example

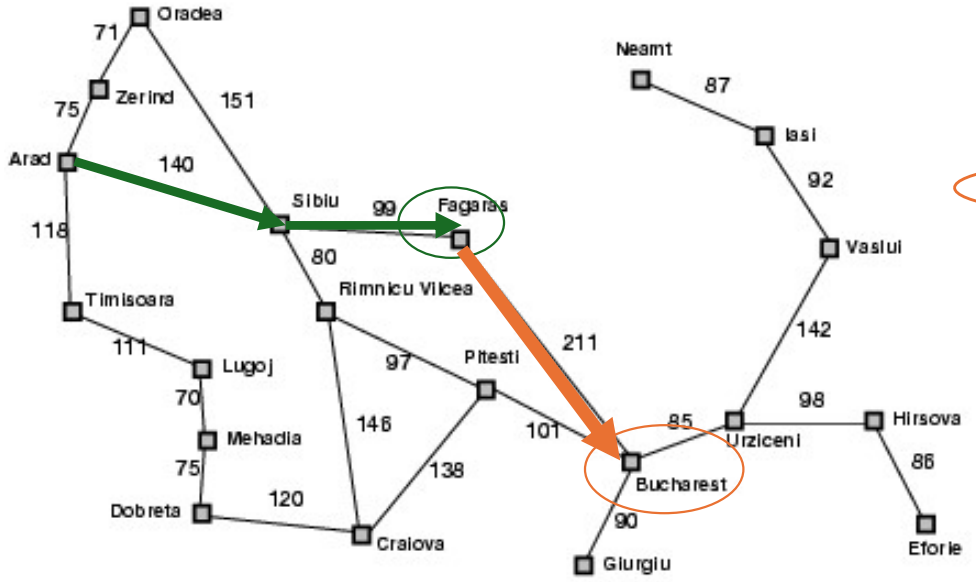




Straight–line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

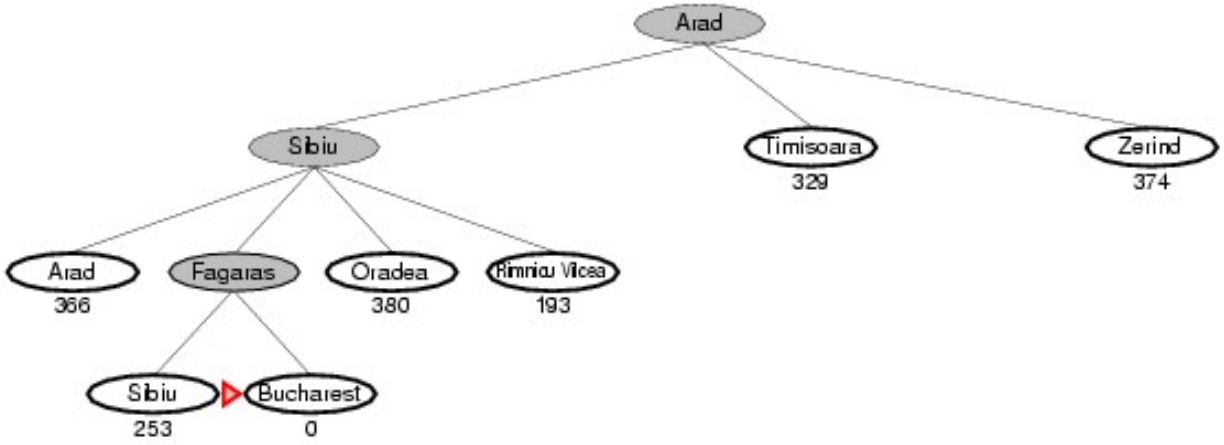
Greedy Best-First Search Example





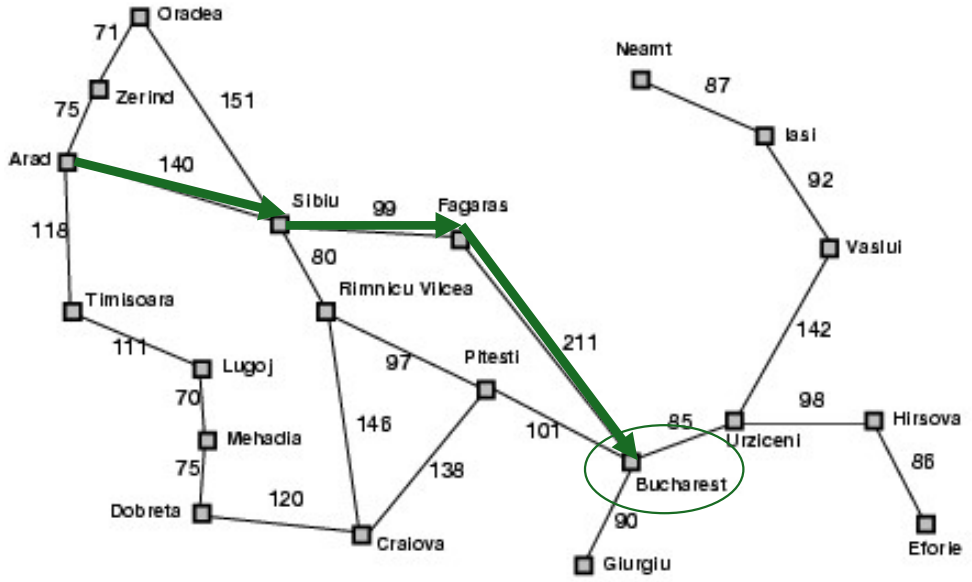
Straight–line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Greedy Best-First Search Example



Total:

140 + 99 + 211 = 450 miles



Straight–line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

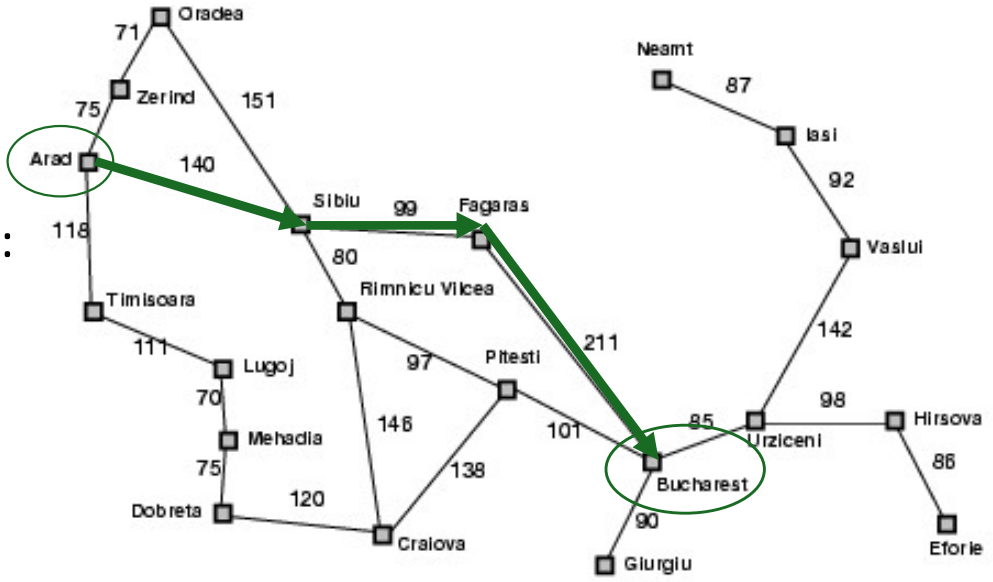
Properties of Greedy Best-First Search

- **Complete?**
  Yes – Best-first search if complete in finite spaces.

- **Optimal?**
  No

Total:
140 + 99 + 211 = 450 miles

Alternative through Rimnicu Vilcea:
140 + 80 + 97 + 101 = 418 miles



Straight–line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Properties of Greedy Best-First Search

- **Complete?**
  Yes – Best-first search is complete in finite spaces.

- **Optimal?**
  No

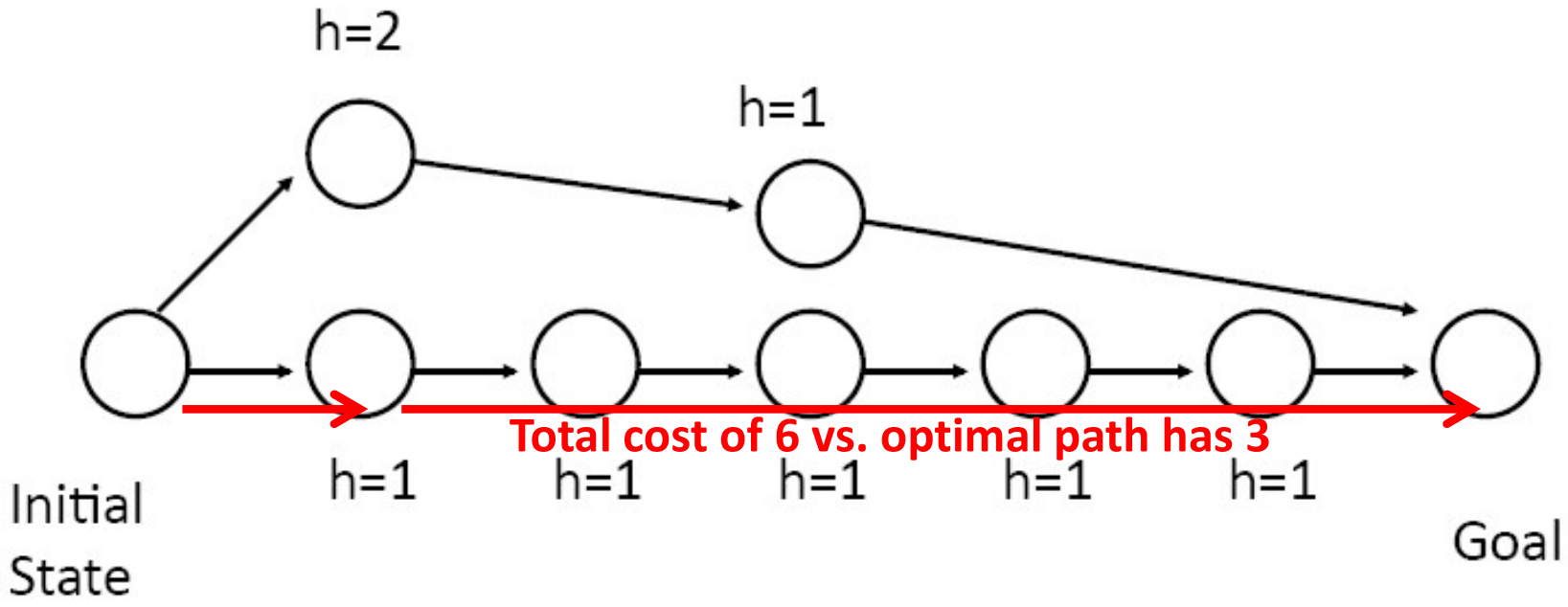
d: depth of the optimal solution
m: max. depth of tree
b: maximum branching factor

- **Time?**
  Worst case: $O(b^m)$ ⇔ like DFS
  Best case: $O(bm)$ – If $h(n)$ is 100% accurate we only expand a single path.

- **Space?**
  Same as time complexity. The worst case is very bad!

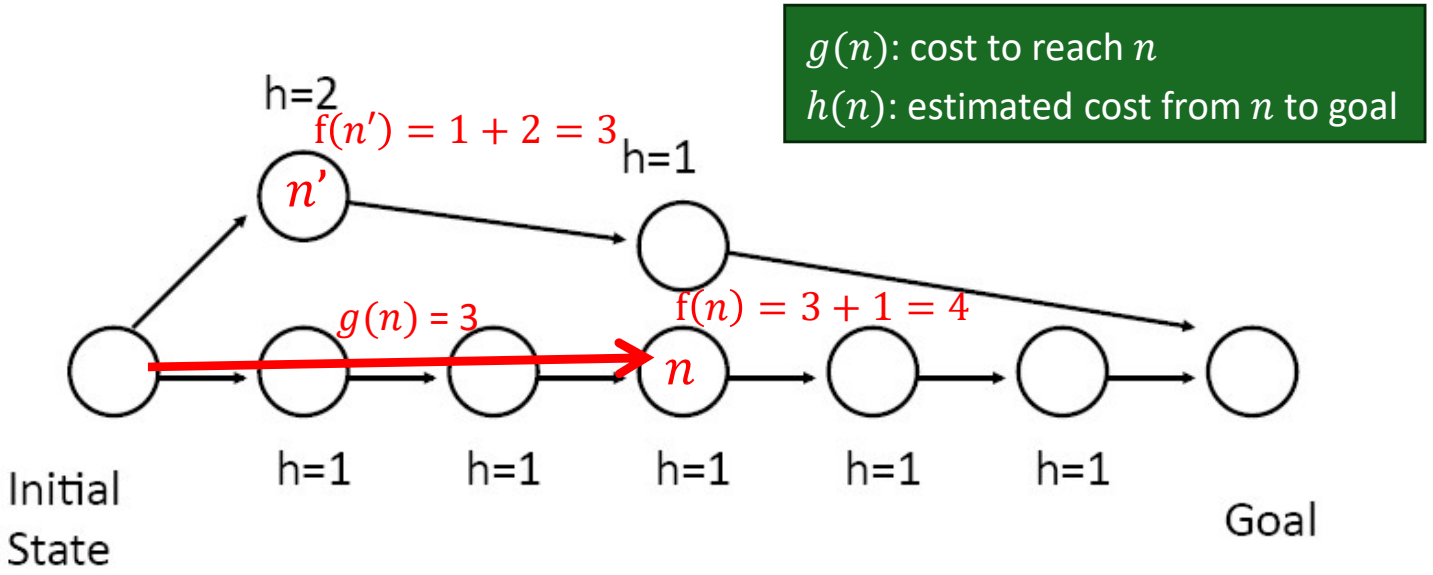
The Optimality Problem of Greedy Best-First search

Greedy best-first search only considers the estimated cost to the goal: $h(n)$



$h = 1$ is better than $h = 2$.
Greedy means it will never reconsider a decision!

A* Search



- **Idea**: Consider the complete path for planning.
- The evaluation function $f(n)$ is the estimated total cost of the path through node n to the goal:

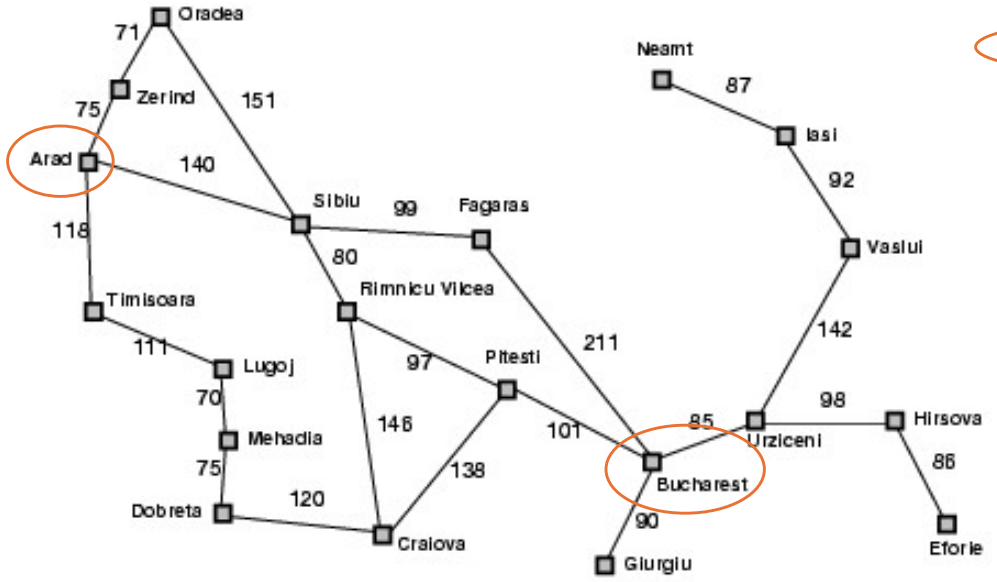
$$f(n) = g(n) + h(n)$$

- Best-first search computes $f()$ for all nodes in the frontier. In the example above, n and n' are in the frontier. The search will expand n' with $f(n') = 3$ before n with $f(n) = 4$ and find the optimal path.

A* Search Example

Expansion rule: $f(n) = g(n) + h(n) =$ Arad 366=0+366

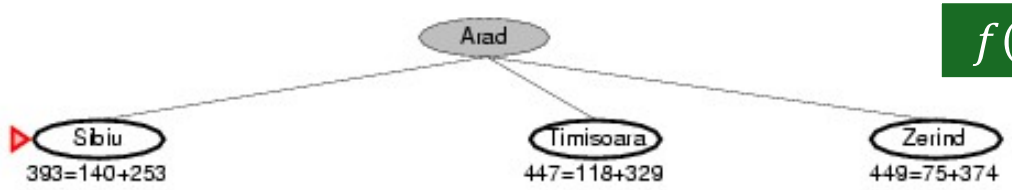
Expand the node with the smallest f(n)



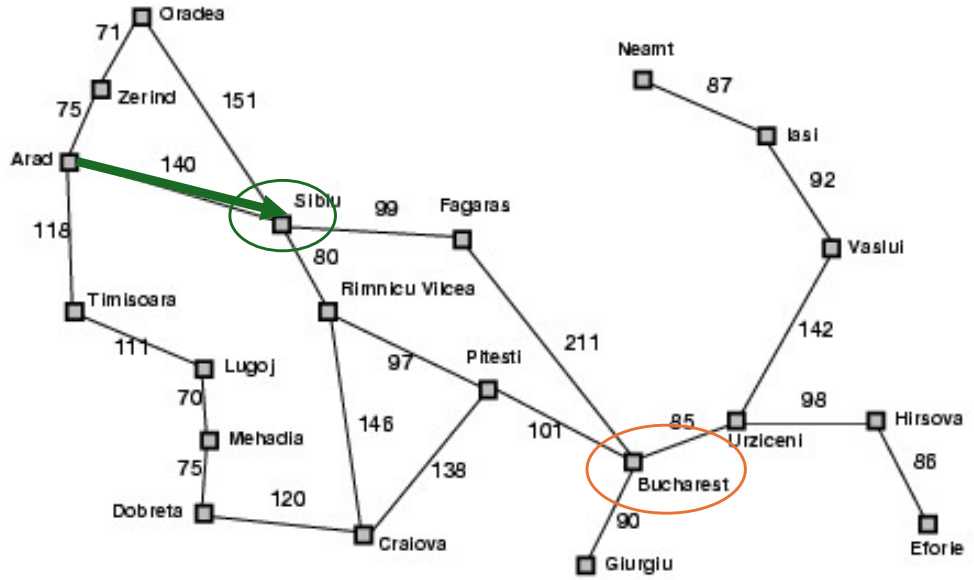
$h(n)$

Straight–line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

A* Search Example



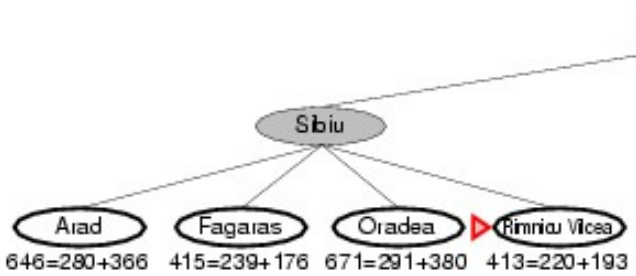
$$f(n) = g(n) + h(n)$$



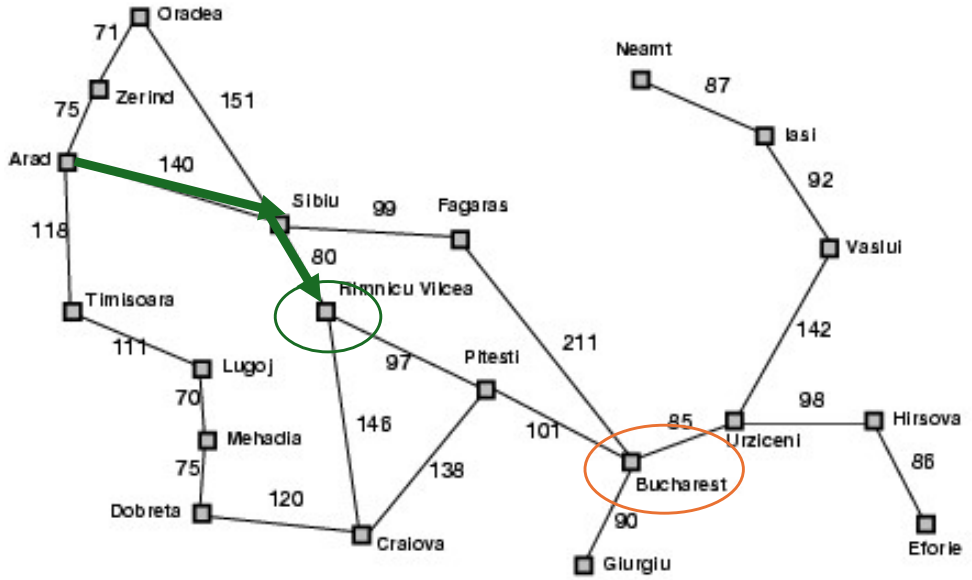
$h(n)$

Straight–line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

A* Search Example



$$f(n) = g(n) + h(n)$$

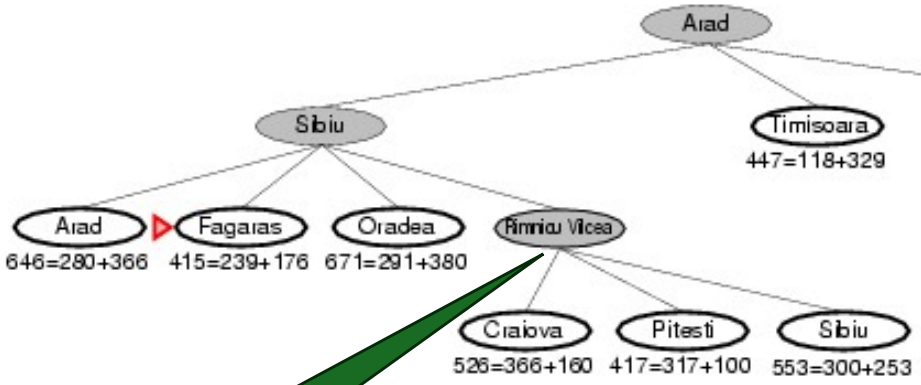


$h(n)$

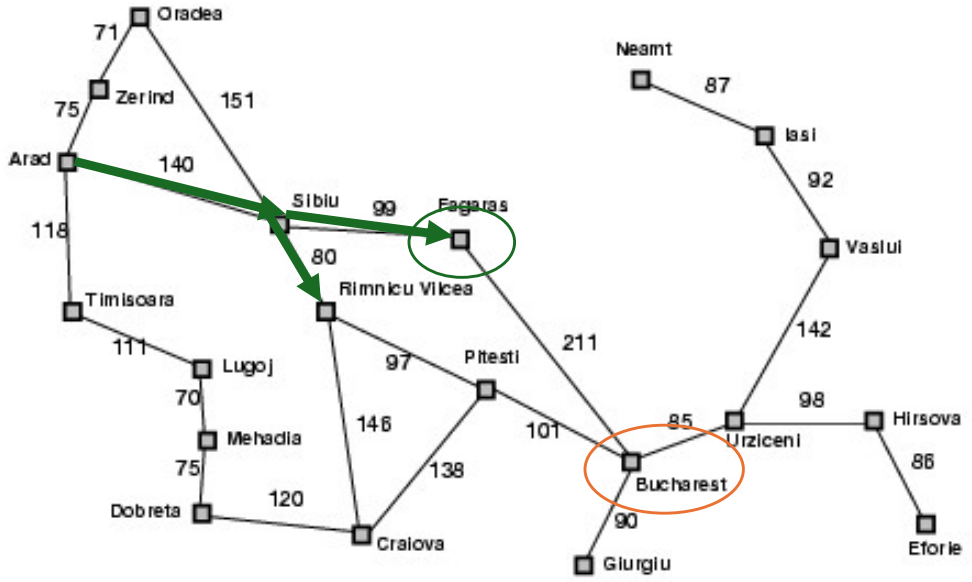
Straight–line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

A* Search Example

$$f(n) = g(n) + h(n)$$



Reconsiders Rimnicu Vilcea because Fagaras may have a shorter total cost to Bucharest.

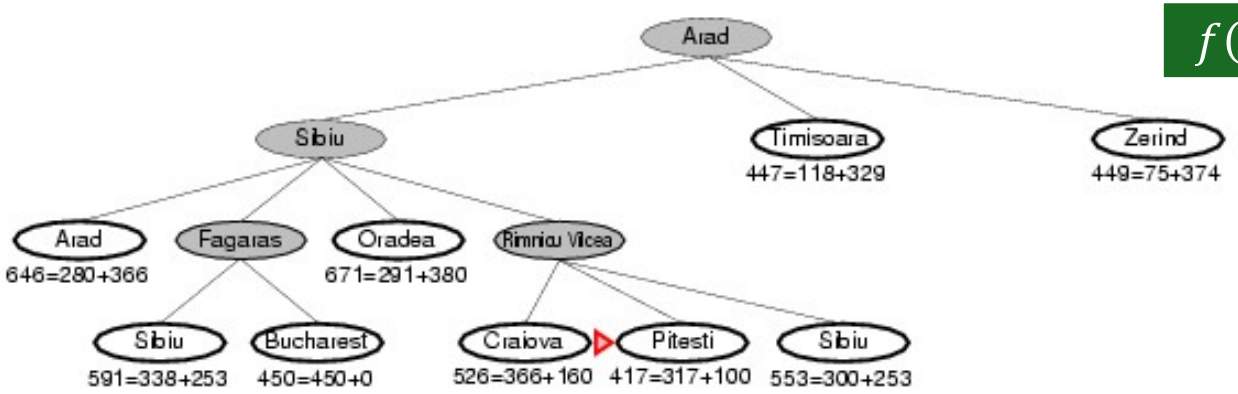


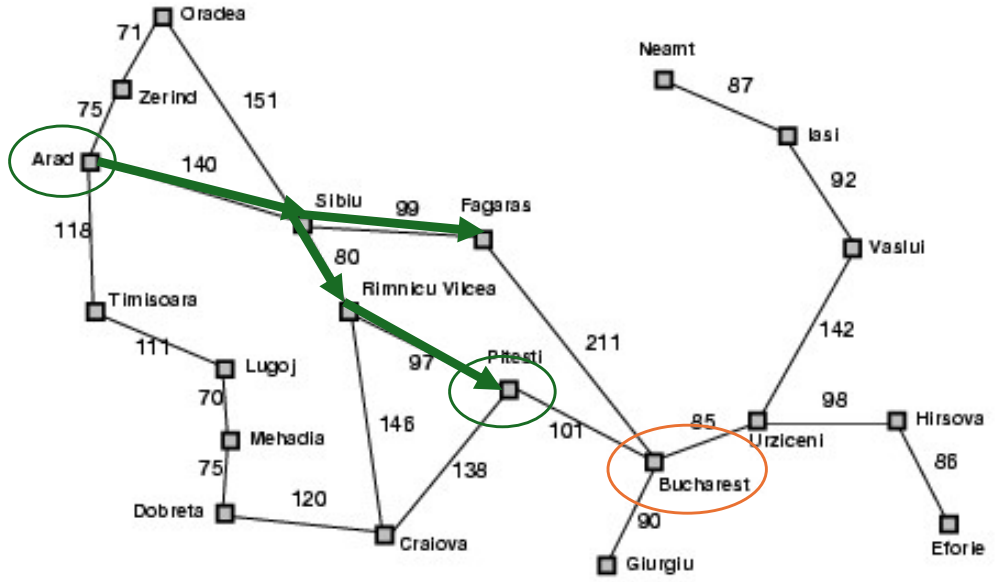
$h(n)$

Straight–line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

A* Search Example

$$f(n) = g(n) + h(n)$$



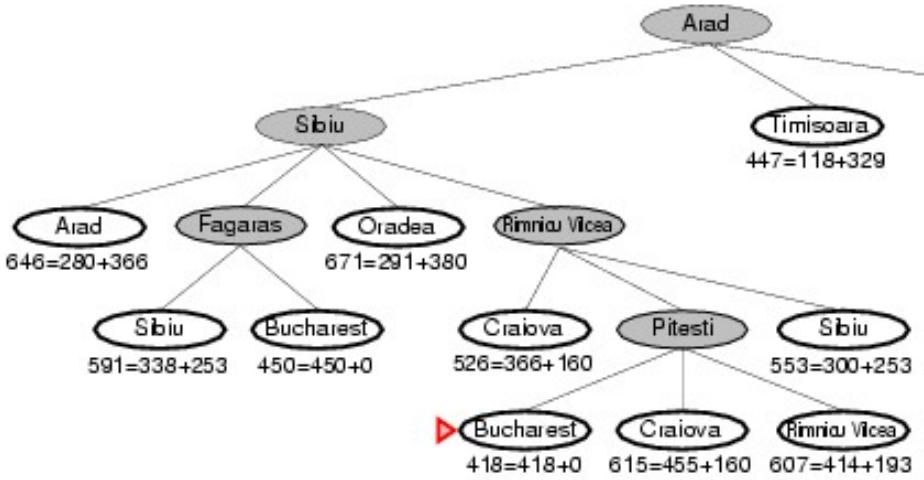


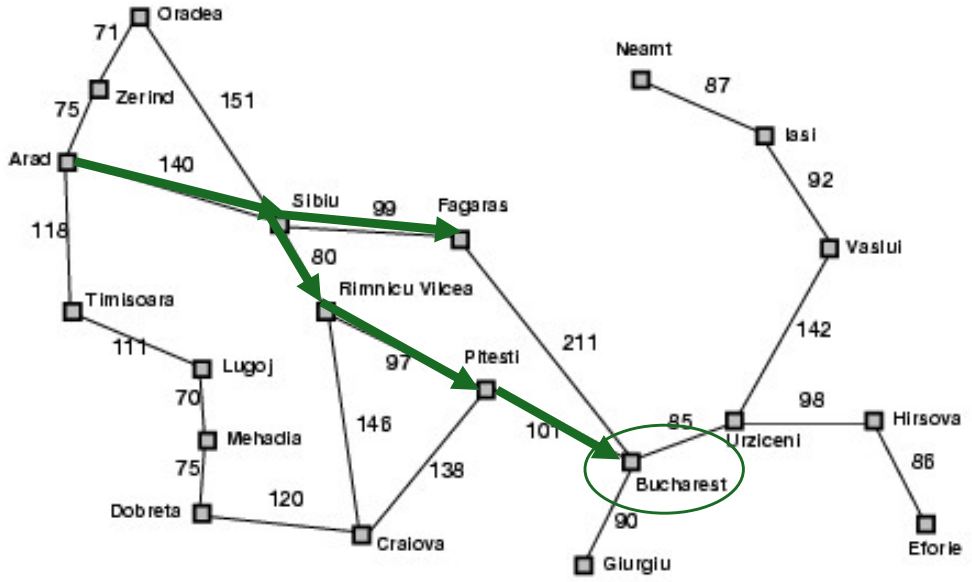
$h(n)$

Straight–line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

A* Search Example

$$f(n) = g(n) + h(n)$$





$h(n)$

Straight–line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Admissible Heuristics and Optimality

an admissible heuristic is a **lower bound** (an optimistic estimate) and never overestimates the true cost to reach the goal.

Definition: A heuristic h is **admissible** if for every node n, $h(n) \leq h^*(n)$, where $h^*(n)$ is the true cost to reach the goal state from n.

Example: The straight-line distance never overestimates the actual road distance on a map.

Theorem: If h is admissible, A* is **optimal**.

Proof: The admissible heuristic forces A* to explore all possible shorter paths before it reaches the goal state.

Theorem: A* **is optimally efficient**. No other tree-based search algorithm that employs the same heuristic can expand fewer nodes and still guarantee the optimal solution.

Proof: Any algorithm that does not expand all nodes with $f(n) < C^*$ (the lowest cost of going to a goal node) cannot be optimal because it risks missing the optimal solution.

Properties of A*Search

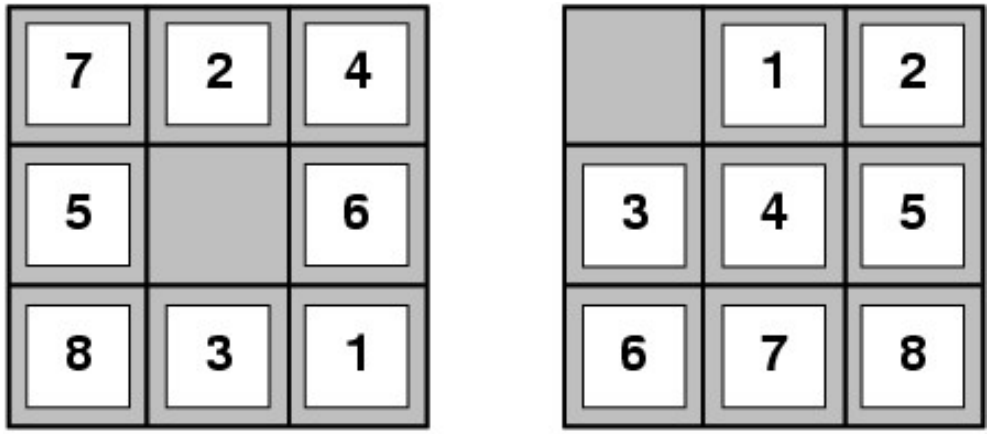
- **Complete?**
  Yes

- **Optimal?**
  Yes

- **Time?**
  Number of nodes for which $f(n) \leq C^*$ in the worst case $O(b^d)$ like BFS.

- **Space?**
  Same as time complexity. This is often too high unless a very good heuristic is know.

Informed Search Designing Heuristic Functions



Example heuristics for the 8-puzzle:

- $h_1(n)$ = number of misplaced tiles
- $h_1(start) = 8$

- $h_2(n)$ = total Manhattan distance (number of moves from the desired location of each tile)
- $h_2(start) = 3 + 1 + 2 + 2 + 2 + 3 + 3 + 2 = 18$

1 needs to move 3 positions

Are h_1 and h_2 admissible?

Early vs. Late Goal Testing

- **Early Goal Testing :** The goal is checked **immediately when a successor is generated**.
- **Late Goal Testing:** The node is first placed in the frontier. We test for the goal **only when it is selected and removed from the frontier**.

Search Algorithm	Goal Testing	Reason
BFS	**Early**	First generated goal is shallowest when step costs are equal
DFS	**Early commonly**	Can stop immediately when goal is found
Depth-Limited Search	**Early commonly**	Goal can be detected during successor processing
Iterative Deepening (IDS)	**Early commonly**	Based on depth-limited DFS
Uniform-Cost Search (UCS)	**Late**	Must ensure lowest-cost node is selected first, g(n)
Greedy Best-First Search (GBFS)	**Early or Late. No significant Difference**	Frontier selection is based on (h(n))
A*	**Late**	Goal should be accepted when selected with minimum (f(n)=g(n)+h(n))